

Lecture 10: Genealogies and phylodynamics with phylopomp

Qianying (Ruby) Lin Spencer J. Fox Jesse Wheeler

Objectives

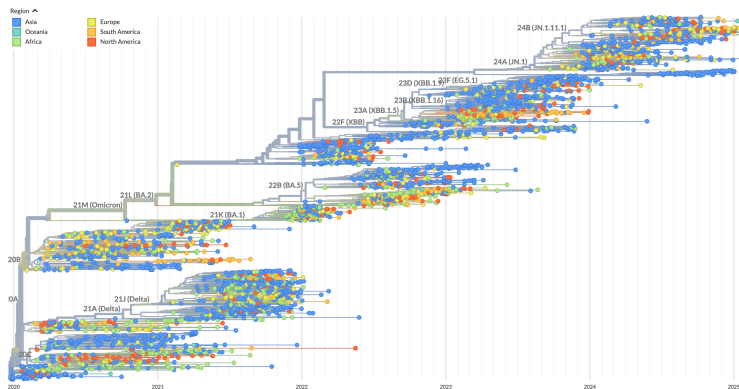
So far the **data** were a time series of case counts. Now the data are a **genealogy**: a family tree of the pathogen, reconstructed from sampled genome sequences.

By the end of this lecture you will be able to:

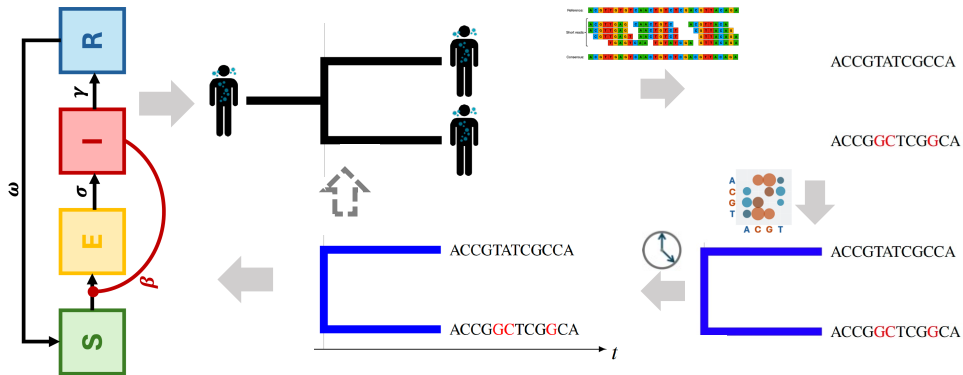
1. Say what a genealogy is, and how it links genome data to the transmission dynamics.
2. Simulate a genealogy from a compartmental model with `phylomp`.
3. Read a tree and its **lineages-through-time**, and see the epidemic in their shape.
4. Evaluate the **likelihood** of a model given a genealogy, using the *same* particle filter from Lectures A and B.

The pathogen carries its own history

As an epidemic spreads, the pathogen mutates from host to host. Sequencing the samples, then reconstructing their phylogeny with tools built for it (e.g. BEAST), yields a **genealogy**: an estimate of who infected whom, as a tree. `phyloomp` begins where that pipeline ends, taking the tree as its data.

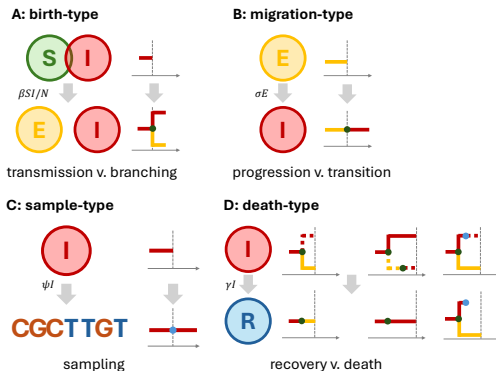


Phylogenetics asks *what*; phylodynamics asks *why*



phyloPomp treats the genealogy as a **partial observation** of an epidemic, and infers the transmission dynamics behind it, the same POMP idea as the rest of the module: a hidden process, observed noisily.

The tree is a partial observation



Births (transmissions), deaths (recoveries), and samples are events in the hidden epidemic. We only ever see **some** of them, and the tree is what survives. `phylopomp` takes that genealogy as given, observed exactly; here we **simulate** trees, so the workflow is self-contained. The theory is in King, Lin, and Ionides (2025).

Installing phylopomp

phylopomp lives on GitHub at <https://github.com/kingaa/phylopomp>. Install the latest version, then check it loaded:

```
# install.packages("pak")           # if you do not have it
pak::pak("kingaa/phylopomp")

library(phylopomp)
packageVersion("phylopomp")        # 0.19.4.2 or newer
```

It compiles C++ under the hood, so give it a minute the first time.

Two things it does

phylopomp provides two families of tools.

- ▶ **Simulate** a genealogy from a compartmental model, with one function per model: `runSIR`, `runSEIRS`, `runSIIR` (two strains), `runSI2R` (super-spreading), `runLBDP` (linear birth-death), and more.
- ▶ **Infer**: turn a genealogy into a pomp object with `sir_pomp`, `seirs_pomp`, `si2rs_pomp`, `lbdp_pomp`, ..., and then use everything you already know, `pfilter`, `mif2`, `logmeanexp`.

The inference tools are newer and still growing; a handful of models are wired up so far.

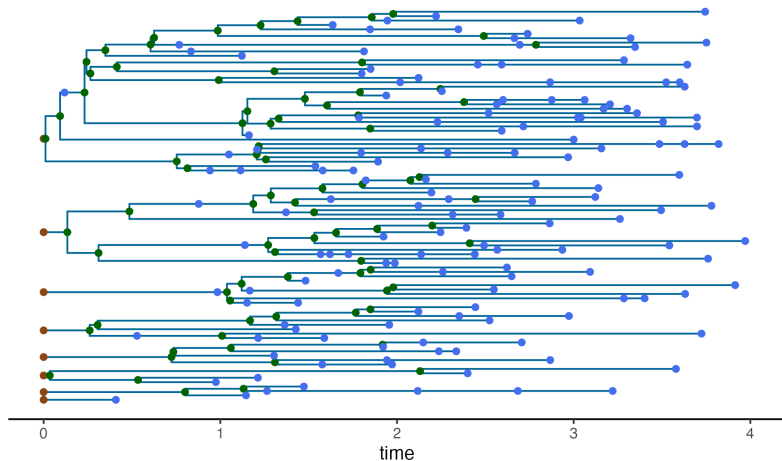
One function builds and simulates the tree

```
library(phylopomp)
set.seed(2024)
runSIR(
  time = 4,           # observation window, t = 0 .. 4
  Beta = 3, gamma = 1, # transmission and recovery rates
  psi = 0.4,          # sampling rate (how many tips you get)
  pop = 500,          # total host population
  S0 = 0.97, I0 = 0.03 # initial FRACTIONS of pop (not counts)
) -> G

plot(G, points = TRUE) # the genealogy
```

Note the modern interface: the initial states are **fractions** of pop, and each model has its own run... function.

A simulated SIR genealogy

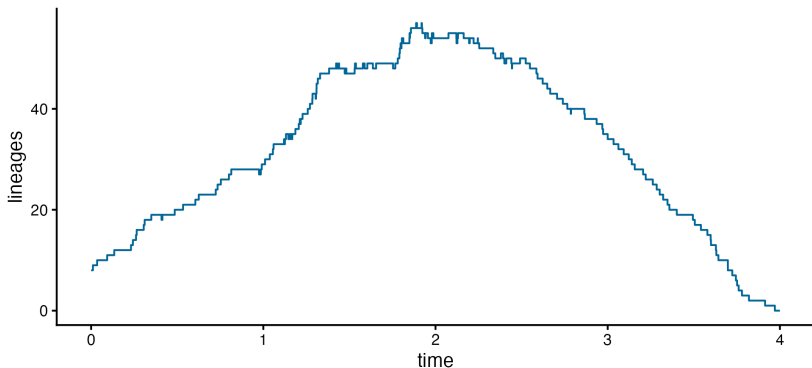


Blue dots are **samples** (the tips), green dots are **branch points** (shared ancestors), brown dots are the **roots**. Time runs left to right.

The tree's shape is the epidemic

`lineages()` counts how many branches are alive at each moment. It rises as the epidemic grows and falls as it burns out.

```
plot(lineages(G))
```



Change the parameters partway through

`continue...` picks up a finished run and keeps going with new parameters, handy for a control measure that drops transmission mid-epidemic.

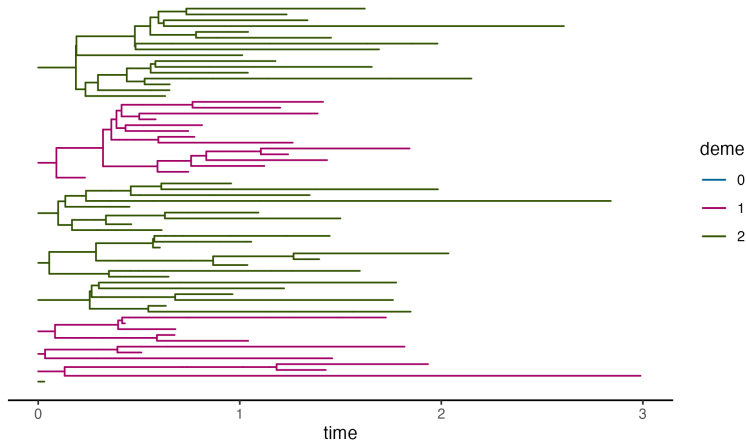
```
runSIR(time = 2, Beta = 3, gamma = 1, psi = 0.5,  
        pop = 500, S0 = 0.98, I0 = 0.02) |>  
  continueSIR(time = 5, Beta = 6, gamma = 2) -> G2    # new rates from t = 2  
plot(G2)
```

Two strains in one tree

runSIIR simulates two competing strains. With `obscure = FALSE`, phylopomp keeps the strain labels and colours each lineage by the strain it belongs to.

```
set.seed(7)
runSIIR(
  time = 3, Beta1 = 4, Beta2 = 8,    # strain 2 spreads faster
  gamma = 1.5, psi1 = 0.6, psi2 = 0.6,
  pop = 300, S_0 = 0.95, I1_0 = 0.03, I2_0 = 0.02
) -> GI
plot(GI, obscure = FALSE)
```

Two strains in one tree



Strain 2 (olive) spreads faster, so it seeds **more, larger** clades than strain 1 (magenta). You can read the competition straight off the tree.

The likelihood of a genealogy

Inference is the same move as the rest of the module. Turn the tree into a pomp object, then hand it to the **particle filter**:

```
library(pomp)
G |>
  sir_pomp(Beta = 3, gamma = 1, psi = 0.4,
           S0 = 0.97, I0 = 0.03, R0 = 0, pop = 500) |>
  pfilter(Np = 1000) |>
  replicate(n = 8) |> concat() -> pf

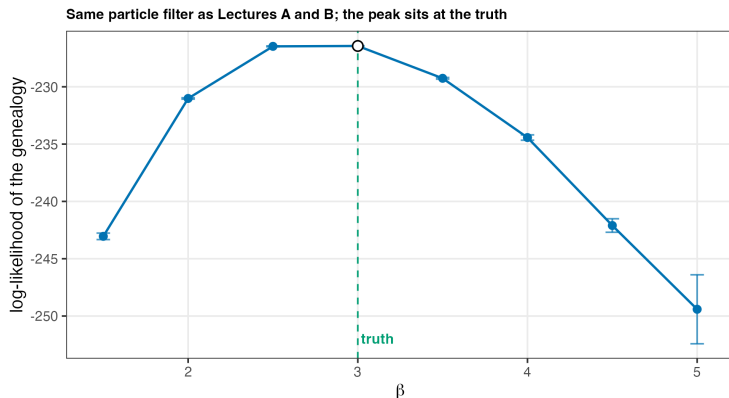
logmeanexp(sapply(pf, logLik), se = TRUE) # same estimator as Lecture 3
```

Nothing here is new. The **data** changed (a tree, not a case count), but $\ell(\theta)$, the particle filter, and `logmeanexp` are exactly as before.

A real, dated tree enters the same pipeline via `parse_newick()`; we simulate here to keep the lecture reproducible.

The likelihood peaks at the truth

We simulated the tree at $\beta = 3$. Scoring a grid of β (all else fixed) with that particle filter puts the peak right where it should be.



From here, `mif2` (Lecture 4) maximises this likelihood exactly as it did for the case-count model.

Your turn

Open `10_phylopomp.Rproj`, then `scripts/EXERCISES.R`. Answers are in `scripts/SOLUTIONS.R`.

1. **Simulate and look.** Build an SIR genealogy, plot the tree and its lineages-through-time, then change the sampling rate `psi` and watch the tree fill in or thin out.
2. **Two strains.** Simulate a two-strain SIIR genealogy and colour it by strain. Which strain leaves more descendants, and why?
3. **Infer.** Fit the SIR model back to its own tree with `pfilter`, sweep `Beta`, and confirm the likelihood peaks near the truth.

A stretch exercise fits the **wrong** model to a tree and compares it, by likelihood, with the model that made it.

Where we've got to

- ▶ A **genealogy** is a partially observed epidemic: the same POMP idea, with a tree for data instead of a case-count series.
- ▶ phylopomp **simulates** trees from compartmental models (`runSIR`, `runSIIR`, ...) and **turns them into pomp objects** (`sir_pomp`, ...).
- ▶ Once it is a pomp object, **everything transfers**: `pfilter` scores it, `logmeanexp` de-biases, `mif2` maximises.
- ▶ The tree's **shape**, its lineages-through-time, carries the dynamics that inference reads.

Genome data, same pipeline. That is the payoff of the POMP framework.

References

King, Aaron A, Qianying Lin, and Edward L Ionides. 2025. “Exact Phylodynamic Likelihood via Structured Markov Genealogy Processes.” *ArXiv*, arXiv–2405.

License, acknowledgments, and links

- ▶ Produced with the `phylopomp` package of Aaron A. King and collaborators (King, Lin, and Ionides 2025).
- ▶ This lesson is prepared for the Simulation-based Inference for Epidemiological Dynamics module at the Summer Institute in Statistics and Modeling in Infectious Diseases, SISMID.
- ▶ The materials build on previous versions of this course and related courses.
- ▶ Licensed under the Creative Commons Attribution-NonCommercial license. Please share and remix non-commercially, mentioning its origin. 
- ▶ Produced with R version 4.5.1, `pomp` version 6.4, and `phylopomp` version 0.19.4.2.

R code for this lesson